



Implementation of a Laundry Management System Using Python and MySQL

Group 11

Names : 1. Mufti Shafiq (2421010104)
2. Nazilatun Natasya Ramadhani (2421010173)

Class : 3CS1

Faculty : 1. Ivan Firdaus
2. Tri Agus Riyadi, S.Kom., M.Kom.

CEP CCIT
FAKULTAS TEKNIK UNIVERSITAS INDONESIA
2025

PROJECT INFORMATION

Project Title : Implementation of a Laundry Management System
Using Python and MySQL

Batch Code : 3CS1

Start Date : September 15, 2025

End Date : September 25, 2025

Names of Faculty : Mr Ivan Firdaus
Mr Tri Agus Riyadi, S.Kom., M.Kom.

Names of Developer : Mufti Shafiq
Nazilatun Natasya Ramadhani

Date of Submission: September 25, 2025

ACKNOWLEDGEMENT

Praise our gratitude to Allah SWT, because for the abundance of grace the authors can complete this paper on time without any significant obstacles and in accordance with expectations.

Our gratitude goes to Mr. Ivan Firdaus, and Mr. Tri Agus Riyadi, S.Kom., M.Kom. as a class lecturer and supervisor who has helped provide direction and understanding in the preparation of this paper.

We realize that in the preparation of this paper there are still many shortcomings due to our limitations. Therefore, the compiler really hopes for criticism and suggestions to improve this paper. Hopefully what is written can be useful for all parties in need.

SYSTEM SUMMARY

This research aims to develop a laundry management system using Python integrated with MySQL. The system is designed to simplify the management of customer data, laundry packages, and transaction records in an efficient and organized way. By utilizing Python as the main programming language and MySQL as the database, this system provides a reliable solution for managing laundry activities, automating transaction calculations, and reducing manual errors. The implementation of this system is expected to improve the accuracy of data processing, enhance service efficiency, and support better management of laundry business operations.

DATABASE DESIGN

Database Name : Laundryy

Number of Tables : 4

- Users
- Packages
- Customers
- Transactions

Users Table : 5

- User_id
- Username
- Password
- Email
- Role

Packages Table : 4

- Package_id
- Package_name
- Price_per_kg
- Estimated_days

DATABASE DESIGN

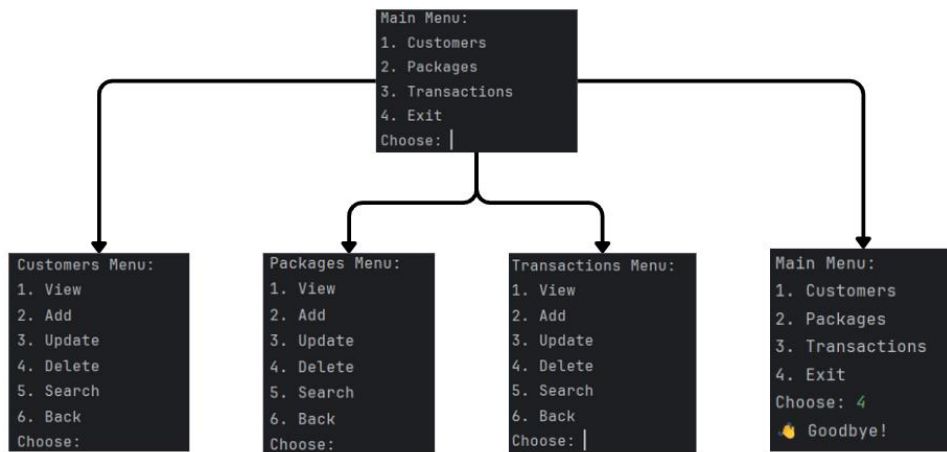
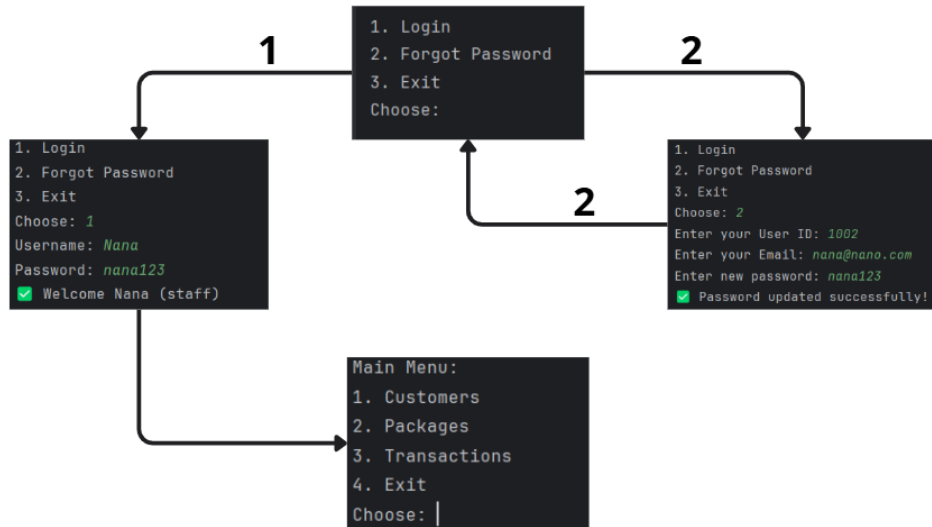
Customers Table : 4

- Customer_id
- Name
- Addres
- Phone_number

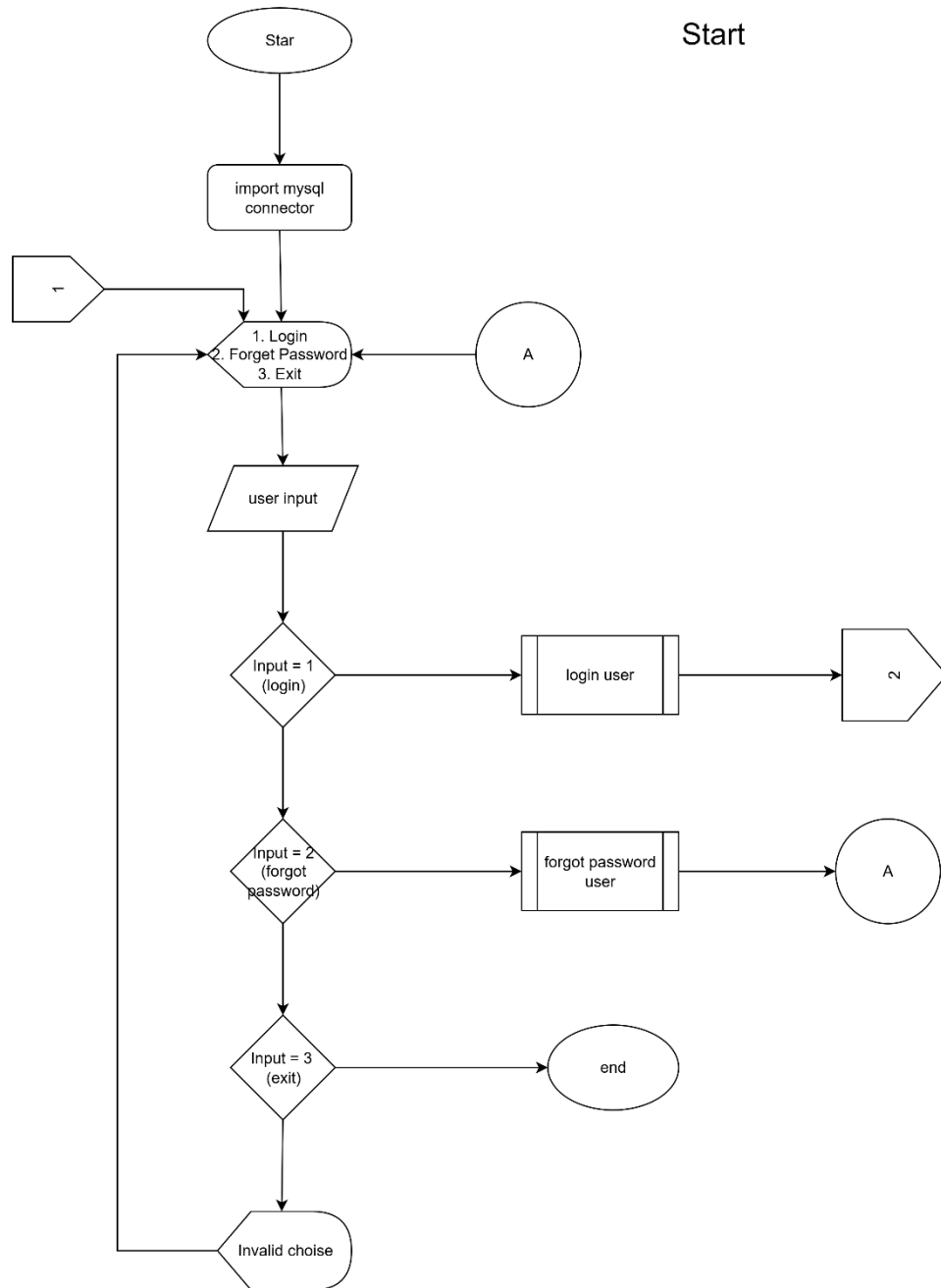
Transactions Table : 9

- Transaction_id
- Customer_id
- Package_id
- Weight_kg
- Total_price
- Date_in
- Date_out
- Status
- Input_user_id

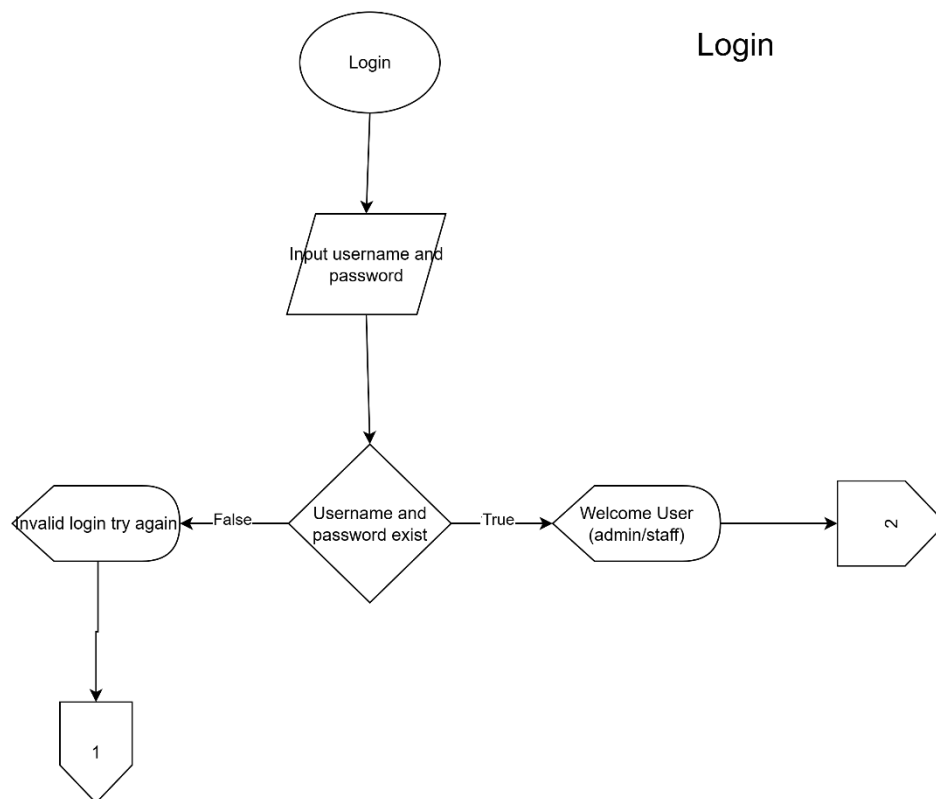
MINDMAP



FLOWCHART

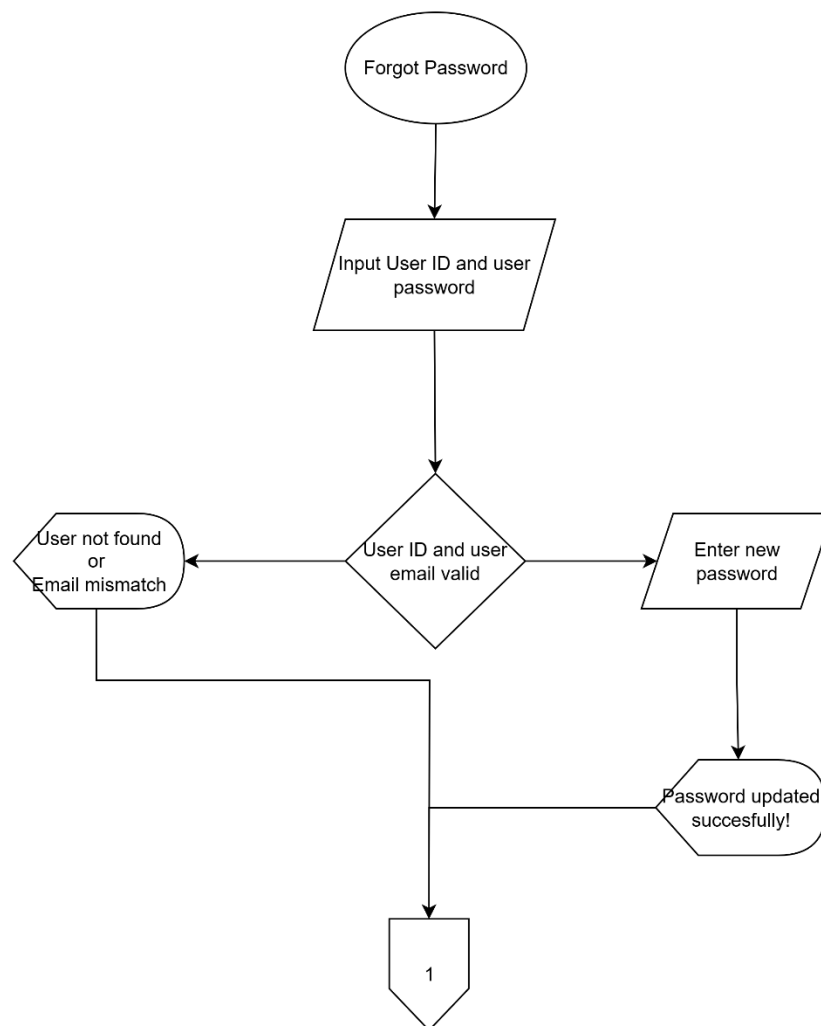


FLOWCHART

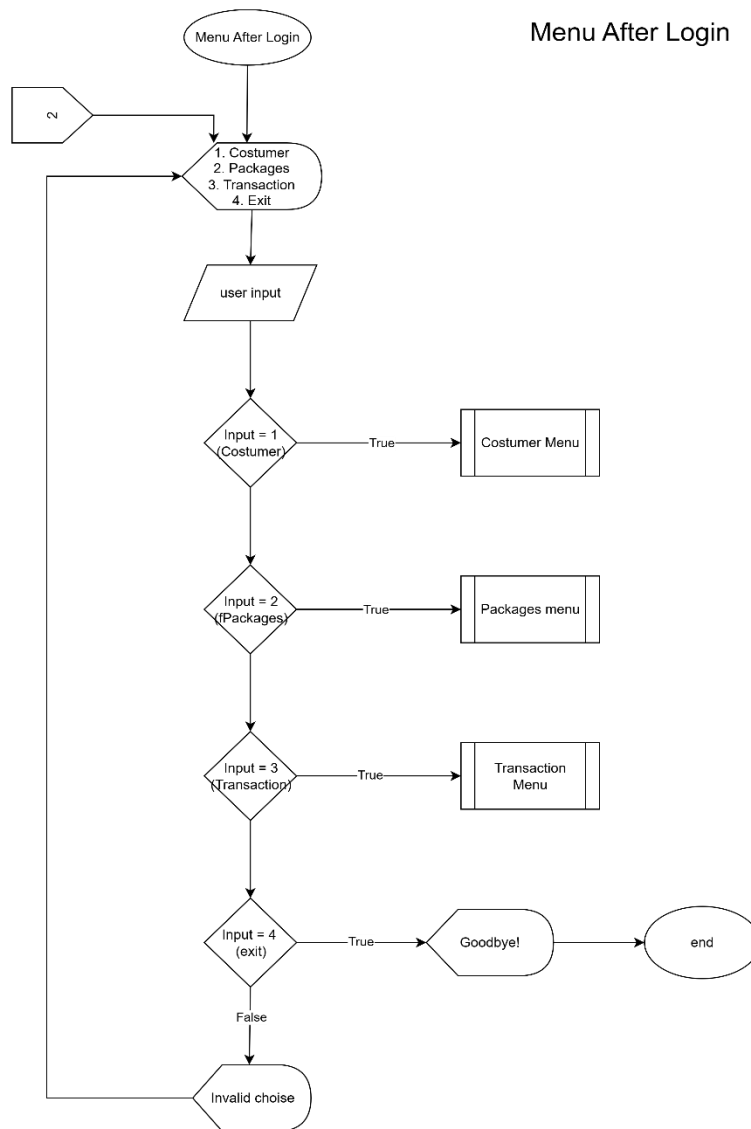


FLOWCHART

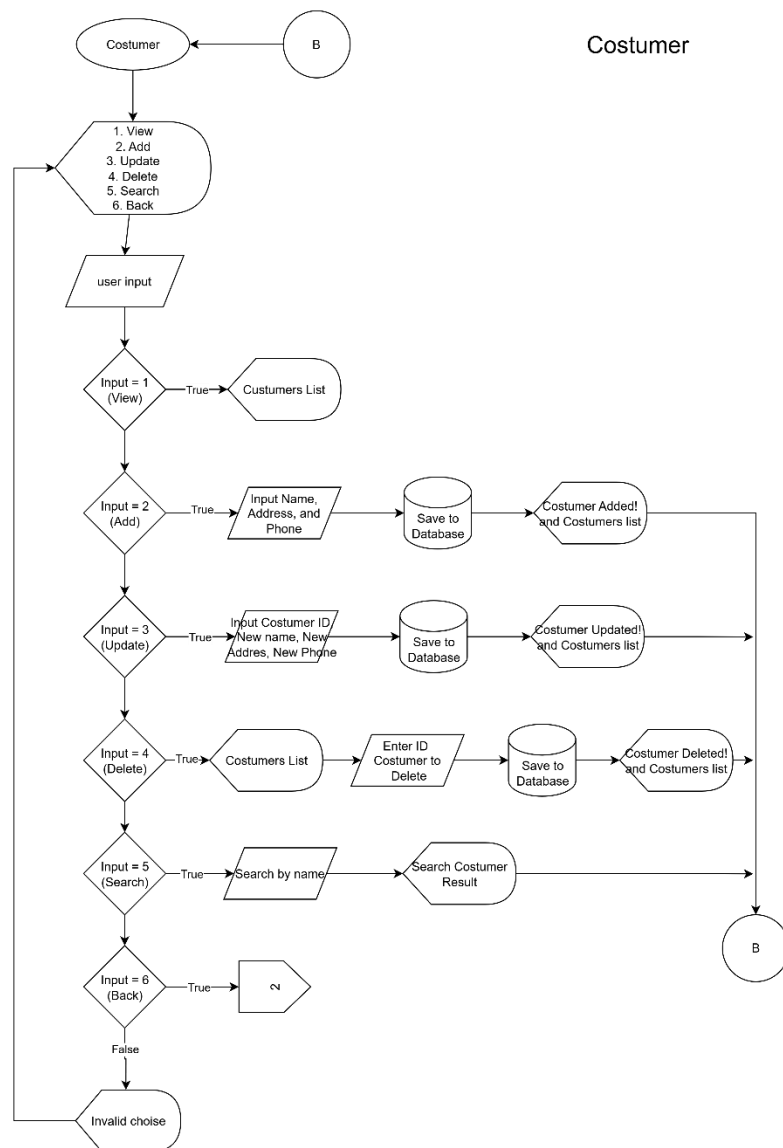
Forgot Password



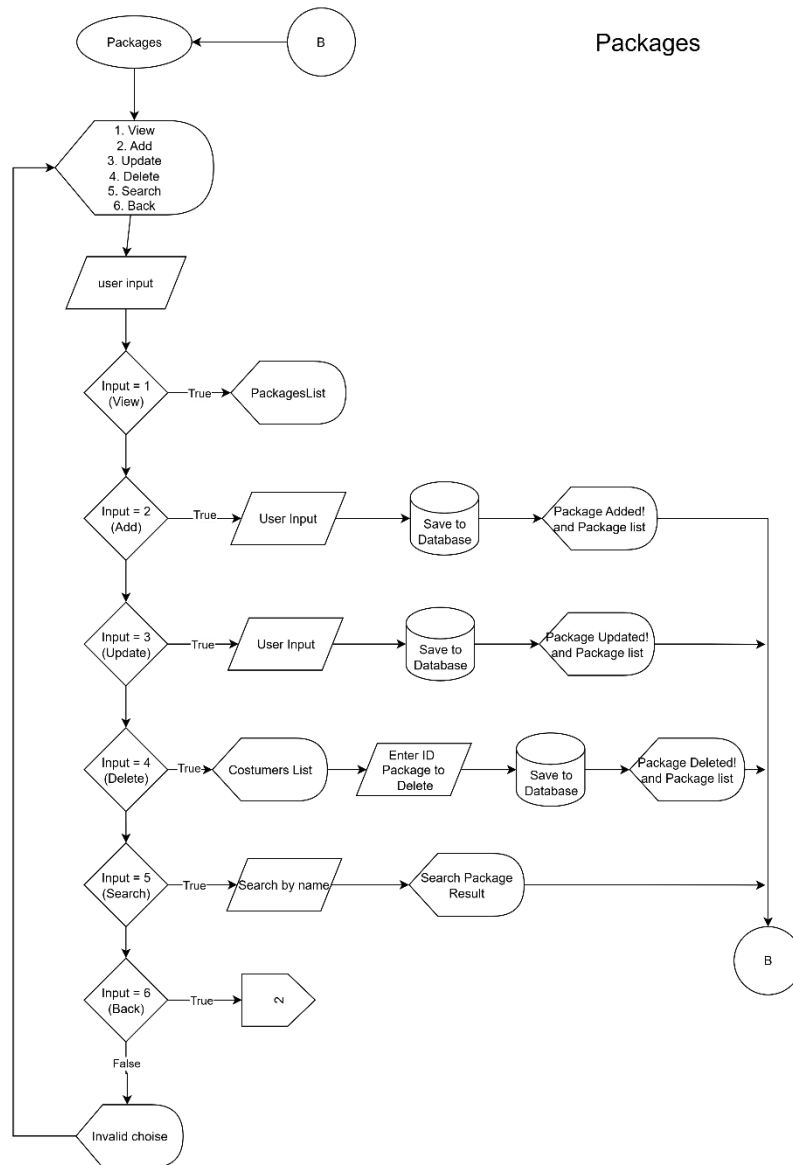
FLOWCHART



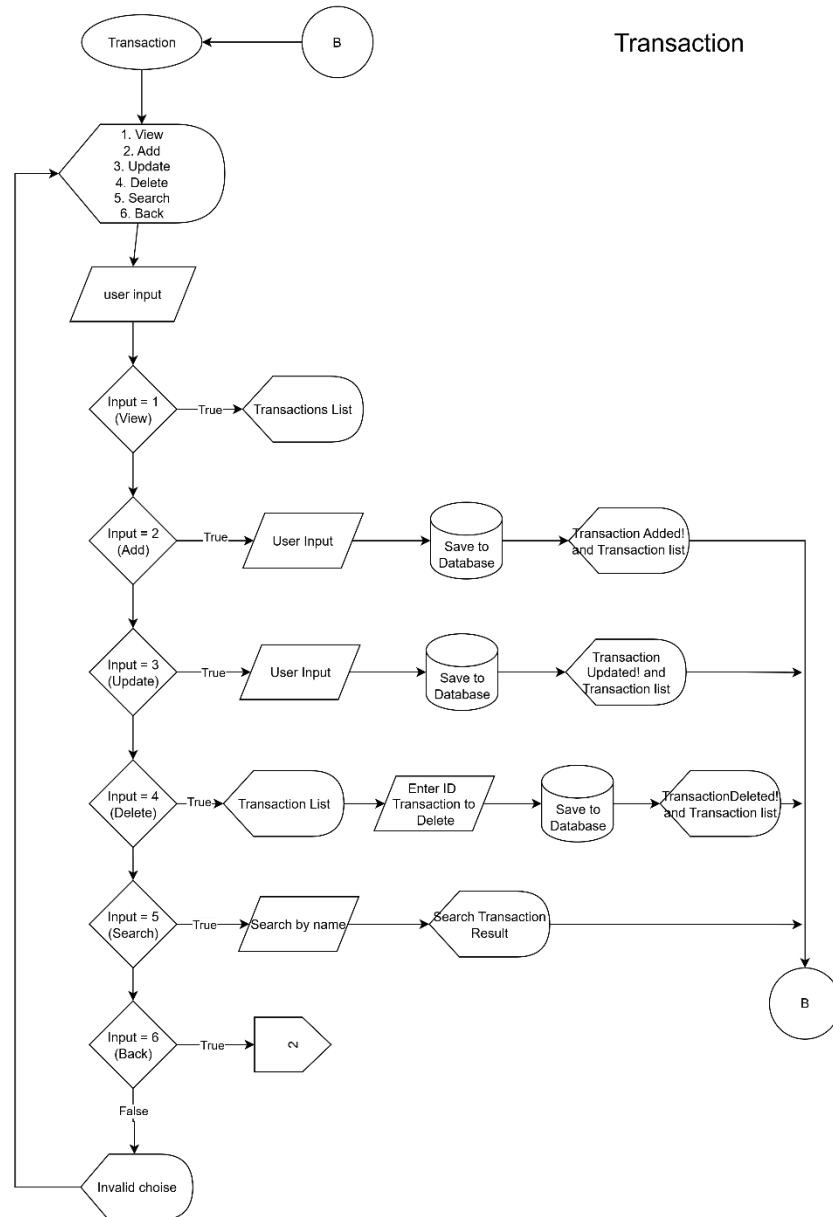
FLOWCHART



FLOWCHART



FLOWCHART



SCHEMATIC DIAGRAM

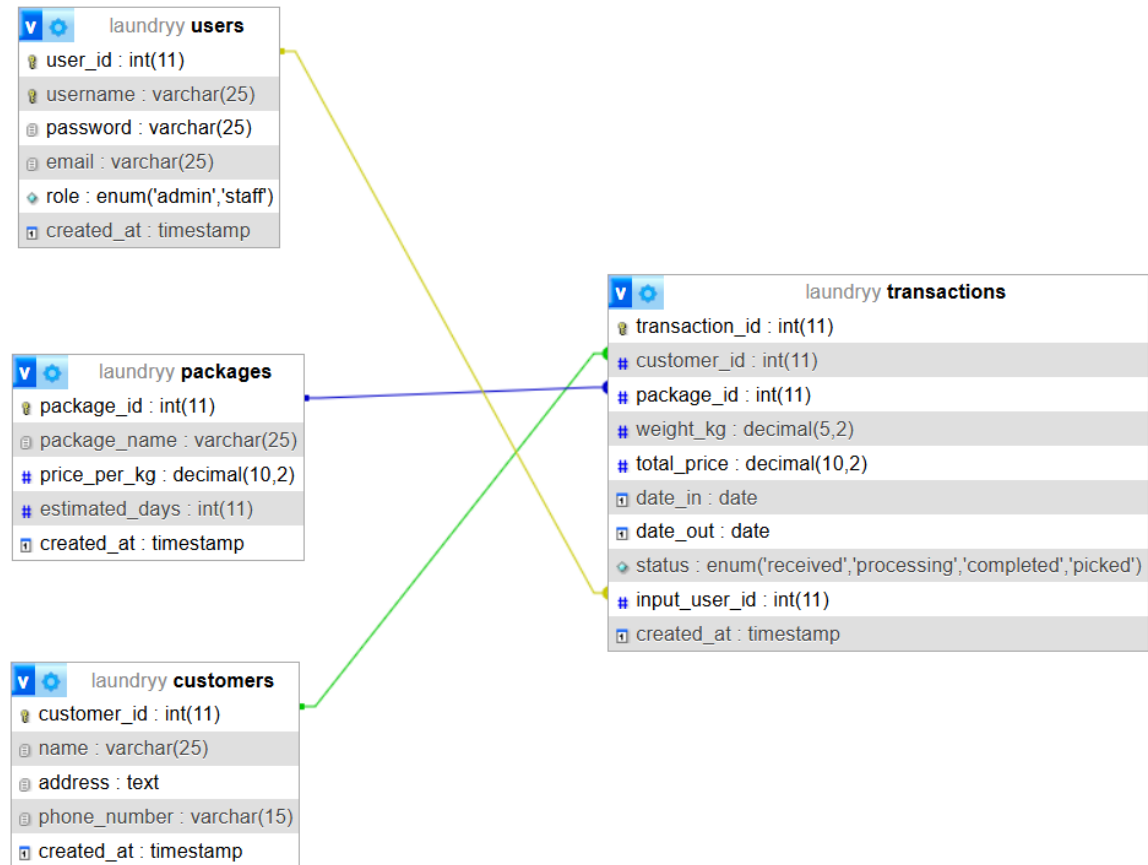


TABLE DESIGN

1. Users Table

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐ 1	user_id 🔑	int(11)			No	None		AUTO_INCREMENT	Change Drop More
☐ 2	username 🔑	varchar(25)	utf8mb4_general_ci		No	None			Change Drop More
☐ 3	password	varchar(25)	utf8mb4_general_ci		No	None			Change Drop More
☐ 4	email	varchar(25)	utf8mb4_general_ci		No	None			Change Drop More
☐ 5	role	enum('admin', 'staff')	utf8mb4_general_ci		No	None			Change Drop More
☐ 6	created_at	timestamp			No	current_timestamp()			Change Drop More

2. Packages Table

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐ 1	package_id 🔑	int(11)			No	None		AUTO_INCREMENT	Change Drop More
☐ 2	package_name 🔑	varchar(25)	utf8mb4_general_ci		No	None			Change Drop More
☐ 3	price_per_kg	decimal(10,2)			No	None			Change Drop More
☐ 4	estimated_days	int(11)			No	None			Change Drop More
☐ 5	created_at	timestamp			No	current_timestamp()			Change Drop More

3. Customers Table

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐ 1	customer_id 🔑	int(11)			No	None		AUTO_INCREMENT	Change Drop More
☐ 2	name	varchar(25)	utf8mb4_general_ci		No	None			Change Drop More
☐ 3	address	text	utf8mb4_general_ci		Yes	NULL			Change Drop More
☐ 4	phone_number	varchar(15)	utf8mb4_general_ci		Yes	NULL			Change Drop More
☐ 5	created_at	timestamp			No	current_timestamp()			Change Drop More

4. Transactions Table

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐ 1	transaction_id 🔑	int(11)			No	None		AUTO_INCREMENT	Change Drop More
☐ 2	customer_id 🔑	int(11)			No	None			Change Drop More
☐ 3	package_id 🔑	int(11)			No	None			Change Drop More
☐ 4	weight_kg	decimal(5,2)			No	None			Change Drop More
☐ 5	total_price	decimal(10,2)			No	None			Change Drop More
☐ 6	date_in	date			No	None			Change Drop More
☐ 7	date_out	date			No	None			Change Drop More
☐ 8	status	enum('received', 'processing', 'completed', 'picked')	utf8mb4_general_ci		Yes	received			Change Drop More
☐ 9	input_user_id 🔑	int(11)			No	None			Change Drop More
☐ 10	created_at	timestamp			No	current_timestamp()			Change Drop More

VALIDATION PERFORMED

1. User Table

column	Method(s) user for Validation	Description
User_ID (PK)	Primary key, NOT NULL	This column contain Primary Key, cannot be empty
Username (FK)	Foreign Key, NOT NULL	This column contain Foreign Key, cannot be empty
Password	NOT NULL	This column cannot be empty
Email	NOT NULL	This column cannot be empty
Role	NULL	This column can be empty

2. Package Table

column	Method(s) user for Validation	Description
Package_ID(PK)	Primary key, NOT NULL	This column contain Primary Key, cannot be empty
Package_name (FK)	Foreign Key, NOT NULL	This column contain Foreign Key, cannot be empty
Price_kg	NOT NULL	This column cannot be empty
Estimated_days	NULL	This column can be empty

3. Customer Table

column	Method(s) user for Validation	Description
Customer_ID(PK)	Primary key, NOT NULL	This column contain Primary Key, cannot be empty
Name	NOT NULL	This column cannot be empty
Address	NOT NULL	This column cannot be empty
Phone_number	NULL	This column can be empty

VALIDATION PERFORMED

4. Transaction Table

coloumn	Method(s) user for Validation	Description
Transaction_ID(PK)	Primary key, NOT NULL	This column contain Primary Key, cannot be empty
Customer_ID(FK)	Foreign Key, NOT NULL	This column cannot be empty
Package_ID(FK)	NOT NULL	This column cannot be empty
Weight_kg	NOT NULL	This column cannot be empty
Total_price	NOT NULL	This column cannot be empty
Date_in	NOT NULL	This column cannot be empty
Date_out	NOT NULL	This column cannot be empty
Status	NULL	This column can be empty
Input_user_ID(FK)	NOT NULL	This column cannot be empty

CONFIGURATION

Get Connection to MySQL

```
import mysql.connector
import datetime

DB_CONFIG = {
    "host": "localhost",
    "user": "root",
    "password": "",
    "database": "laundryy"
}

def get_connection():
    return mysql.connector.connect(**DB_CONFIG)
```

Create Menu for Login & Forgot Password

```
# ===== LOGIN & FORGOT PASSWORD =====
def login():
    while True:
        print("\n1. Login\n2. Forgot Password\n3. Exit")
        choice = input("Choose: ")

        if choice == "1":
            username = input("Username: ")
            password = input("Password: ")

            conn = get_connection()
            cur = conn.cursor(dictionary=True)
            cur.execute("SELECT * FROM users WHERE username=%s AND password=%s", (username, password))
            user = cur.fetchone()
            cur.close()
            conn.close()

            if user:
                print(f"✅ Welcome {user['username']} ({user['role']})")
                return user
            else:
                print("❌ Invalid login. Try again.")

        elif choice == "2":
            forgot_password()
        elif choice == "3":
            return None
        else:
            print("Invalid choice.")

def forgot_password():
    user_id = input("Enter your User ID: ")
    email = input("Enter your Email: ")

    conn = get_connection()
    cur = conn.cursor(dictionary=True)
    cur.execute("SELECT * FROM users WHERE user_id=%s AND email=%s", (user_id, email))
    user = cur.fetchone()

    if not user:
        print("❌ User not found or email mismatch.")
    else:
        new_pass = input("Enter new password: ")
        cur2 = conn.cursor()
        cur2.execute("UPDATE users SET password=%s WHERE user_id=%s", (new_pass, user_id))
        conn.commit()
        cur2.close()
        print("✅ Password updated successfully!")

    cur.close()
    conn.close()
```

SCRIPT

Create CRUD For Customers Menu

```
# ===== CUSTOMERS =====
def view_customers():
    conn = get_connection()
    cur = conn.cursor(dictionary=True)
    cur.execute("SELECT * FROM customers")
    rows = cur.fetchall()
    cur.close()
    conn.close()

    print("\n--- Customers ---")
    for r in rows:
        print(f"{r['customer_id']:>3} | {r['name']:<15} | {r['phone_number'] or '-':<12} | {r['address'] or '-'}")
    print("-----")

def add_customer():
    name = input("Name: ")
    address = input("Address: ")
    phone = input("Phone: ")

    conn = get_connection()
    cur = conn.cursor()
    cur.execute("INSERT INTO customers (name,address,phone_number) VALUES (%s,%s,%s)", (name,address,phone))
    conn.commit()
    cur.close()
    conn.close()
    print("✅ Customer added!")
    view_customers()

def update_customer():
    view_customers()
    cid = input("Enter Customer ID: ")
    name = input("New Name: ")
    address = input("New Address: ")
    phone = input("New Phone: ")

    conn = get_connection()
    cur = conn.cursor()
    cur.execute("UPDATE customers SET name=%s,address=%s,phone_number=%s WHERE customer_id=%s", (name,address,phone,cid))
    conn.commit()
    cur.close()
    conn.close()
    print("✅ Customer updated!")
    view_customers()

def delete_customer():
    view_customers()
    cid = input("Enter Customer ID to delete: ")

    conn = get_connection()
    cur = conn.cursor()
    cur.execute("DELETE FROM customers WHERE customer_id=%s", (cid,))
    conn.commit()
    cur.close()
    conn.close()
    print("✅ Customer deleted!")
    view_customers()

def search_customer():
    keyword = input("Search by name: ")
    conn = get_connection()
    cur = conn.cursor(dictionary=True)
    cur.execute("SELECT * FROM customers WHERE name LIKE %s", (f"%{keyword}%",))
    rows = cur.fetchall()
    cur.close()
    conn.close()

    print("\n--- Search Results ---")
    for r in rows:
        print(f"{r['customer_id']:>3} | {r['name']:<15} | {r['phone_number'] or '-':<12} | {r['address'] or '-'}")
    print("-----")
```

CONFIGURATION

Create CRUD For Packages Menu

```
# ===== PACKAGES =====
def view_packages():
    conn = get_connection()
    cur = conn.cursor(dictionary=True)
    cur.execute("SELECT * FROM packages")
    rows = cur.fetchall()
    cur.close()
    conn.close()

    print("\n--- Packages ---")
    for r in rows:
        print(f"{r['package_id']:>3} | {r['package_name']:<15} | Rp{r['price_per_kg']} | {r['estimated_days']} days")
    print("-----")

def add_package():
    name = input("Package name: ")
    price = float(input("Price per kg: "))
    days = int(input("Estimated days: "))

    conn = get_connection()
    cur = conn.cursor()
    cur.execute("INSERT INTO packages (package_name,price_per_kg,estimated_days) VALUES (%s,%s,%s)", (name,price,days))
    conn.commit()
    cur.close()
    conn.close()
    print("✅ Package added!")
    view_packages()

def update_package():
    view_packages()
    pid = input("Enter Package ID: ")
    name = input("New name: ")
    price = float(input("New price: "))
    days = int(input("New estimated days: "))

    conn = get_connection()
    cur = conn.cursor()
    cur.execute("UPDATE packages SET package_name=%s,price_per_kg=%s,estimated_days=%s WHERE package_id=%s", (name,price,days,pid))
    conn.commit()
    cur.close()
    conn.close()
    print("✅ Package updated!")
    view_packages()

def delete_package():
    view_packages()
    pid = input("Enter Package ID to delete: ")

    conn = get_connection()
    cur = conn.cursor()
    cur.execute("DELETE FROM packages WHERE package_id=%s", (pid,))
    conn.commit()
    cur.close()
    conn.close()
    print("✅ Package deleted!")
    view_packages()

def search_package():
    keyword = input("Search package: ")
    conn = get_connection()
    cur = conn.cursor(dictionary=True)
    cur.execute("SELECT * FROM packages WHERE package_name LIKE %s", (f"%{keyword}%",))
    rows = cur.fetchall()
    cur.close()
    conn.close()

    print("\n--- Search Results ---")
    for r in rows:
        print(f"{r['package_id']:>3} | {r['package_name']:<15} | Rp{r['price_per_kg']} | {r['estimated_days']} days")
    print("-----")
```

SCRIPT

Create CRUD For Transactions Menu

```
# ***** TRANSACTIONS *****
def view_transactions():
    conn = get_connection()
    cur = conn.cursor(dictionary=True)
    cur.execute("""
        SELECT t.transaction_id, c.name AS customer, p.package_name, t.weight_kg, t.total_price,
               t.date_in, t.date_out, t.status, u.username AS staff
        FROM transactions t
        JOIN customers c ON t.customer_id=c.customer_id
        JOIN packages p ON t.package_id=p.package_id
        JOIN users u ON t.input_user_id=u.user_id
    """)
    rows = cur.fetchall()
    cur.close()
    conn.close()

    print("\n--- Transactions ---")
    for r in rows:
        print(f"{r['transaction_id']:>3} | {r['customer']:<12} | {r['package_name']:<12} | {r['weight_kg']}kg | Rp{r['total_price']} | {r['date_in']}+{r['date_out']} | {r['status']} | {r['staff']}"
              "\n-----")

def add_transaction(user):
    view_customers()
    cid = input("Customer ID: ")
    view_packages()
    pid = input("Package ID: ")
    weight = float(input("Weight (kg): "))

    conn = get_connection()
    cur = conn.cursor(dictionary=True)
    cur.execute("SELECT * FROM packages WHERE package_id=%s", (pid,))
    pkg = cur.fetchone()

    if not pkg:
        print("❌ Package not found.")
        return

    total = weight * float(pkg['price_per_kg'])
    date_in = datetime.date.today()
    date_out = date_in + datetime.timedelta(days=int(pkg['estimated_days']))

    cur2 = conn.cursor()
    cur2.execute("""
        INSERT INTO transactions (customer_id,package_id,weight_kg,total_price,date_in,date_out,status,input_user_id)
        VALUES (%s,%s,%s,%s,%s,%s,%s,%s)
    """, (cid,pid,weight,total,date_in,date_out,'received',user['user_id']))
    conn.commit()
    cur2.close()
    conn.close()

    print(f"✅ Transaction added! Total = Rp{total}")
    view_transactions()

def update_transaction():
    view_transactions()
    tid = input("Transaction ID: ")
    status = input("New Status (received/processing/completed/picked_up): ")

    conn = get_connection()
    cur = conn.cursor()
    cur.execute("UPDATE transactions SET status=%s WHERE transaction_id=%s", (status,tid))
    conn.commit()
    cur.close()
    conn.close()

    print("✅ Transaction updated!")
    view_transactions()

def delete_transaction():
    view_transactions()
    tid = input("Transaction ID to delete: ")

    conn = get_connection()
    cur = conn.cursor()
    cur.execute("DELETE FROM transactions WHERE transaction_id=%s", (tid,))
    conn.commit()
    cur.close()
    conn.close()

    print("✅ Transaction deleted!")
    view_transactions()

def search_transaction():
    keyword = input("Search by customer name: ")
    conn = get_connection()
    cur = conn.cursor(dictionary=True)
    cur.execute("""
        SELECT t.transaction_id,c.name,p.package_name,t.total_price,t.status
        FROM transactions t
        JOIN customers c ON t.customer_id=c.customer_id
        JOIN packages p ON t.package_id=p.package_id
        WHERE c.name LIKE %s
    """, (f"%{keyword}%",))
    rows = cur.fetchall()
    cur.close()
    conn.close()

    print("\n--- Search Results ---")
    for r in rows:
        print(f"{r['transaction_id']:>3} | {r['name']:<12} | {r['package_name']:<12} | Rp{r['total_price']} | {r['status']}"
              "\n-----")
```

SCRIPT

Create Main Menu

```
# ===== MAIN =====
def main():
    user = login()
    if not user: return

    while True:
        print("\nMain Menu:\n1. Customers\n2. Packages\n3. Transactions\n4. Exit")
        choice = input("Choose: ")

        if choice == "1":
            while True:
                print("\nCustomers Menu:\n1. View\n2. Add\n3. Update\n4. Delete\n5. Search\n6. Back")
                sub = input("Choose: ")
                if sub == "1": view_customers()
                elif sub == "2": add_customer()
                elif sub == "3": update_customer()
                elif sub == "4": delete_customer()
                elif sub == "5": search_customer()
                elif sub == "6": break
            elif choice == "2":
                while True:
                    print("\nPackages Menu:\n1. View\n2. Add\n3. Update\n4. Delete\n5. Search\n6. Back")
                    sub = input("Choose: ")
                    if sub == "1": view_packages()
                    elif sub == "2": add_package()
                    elif sub == "3": update_package()
                    elif sub == "4": delete_package()
                    elif sub == "5": search_package()
                    elif sub == "6": break
            elif choice == "3":
                while True:
                    print("\nTransactions Menu:\n1. View\n2. Add\n3. Update\n4. Delete\n5. Search\n6. Back")
                    sub = input("Choose: ")
                    if sub == "1": view_transactions()
                    elif sub == "2": add_transaction(user)
                    elif sub == "3": update_transaction()
                    elif sub == "4": delete_transaction()
                    elif sub == "5": search_transaction()
                    elif sub == "6": break
            elif choice == "4":
                print("👋 Goodbye!")
                break

if __name__ == "__main__":
    main()
```

SYSTEM CONFIGURATION

Hardware :

1. HP Laptop 15

Operating System:

1. Windows Server 11

Software :

1. Pycharm
2. MySQL
3. Ms. Word
4. Power Point
5. Github ([project-1/projecto at main · nacnachaa/project-1](#))

PROJEK FILE DETAILS

No	File Name	Remarks
1.	PROJECT1_3CS1 .pdf	Paper File
2.	PPT Project 1-3CS1.pptx	Presentation File